

re

REGULAR EXPRESSIONS

in Natural Language Processing

Finding, cleaning, and extracting patterns in text — one symbol at a time

```
pattern = r"\d{3}-\d{4}"
```

What Is a Regular Expression?

- A regular expression (regex) is a sequence of characters that defines a search pattern.
- It lets you match, locate, extract, or replace text based on rules — not just exact strings.
- Think of it as a “smart find”: instead of searching for one exact word, you search for a shape a piece of text can take.

Pattern in action

`c.t`

cat

✓

cot

✓

cut

✓

cent

✗

“.” matches any single character, so `c.t` matches any 3-letter word starting with `c` and ending with `t`.

Why Regex Matters in NLP



Tokenization

Splitting raw text into words, punctuation, and symbols before any modeling begins.



Text Cleaning

Stripping HTML tags, extra whitespace, special characters, and other noise.



Information Extraction

Pulling out emails, phone numbers, dates, hashtags, and prices from free text.



Validation

Checking that user input (usernames, IDs, formats) follows an expected shape.

Basic Syntax Cheat Sheet

Symbol	Meaning	Example	Matches
.	Any single character	<code>.at</code>	cat, bat, hat
*	0 or more of the previous item	<code>ab*</code>	a, ab, abbb
+	1 or more of the previous item	<code>ab+</code>	ab, abbb (not a)
?	0 or 1 of the previous item (optional)	<code>colou?r</code>	color, colour
[]	Any one character from a set	<code>[aeiou]</code>	a, e, i, o, u
	OR — alternation	<code>cat dog</code>	cat, dog

Character Classes & Shorthands

Shorthand	Same as	Meaning
<code>\d</code>	<code>[0-9]</code>	Any digit
<code>\w</code>	<code>[A-Za-z0-9_]</code>	Any “word” character
<code>\s</code>	<code>[\t\n]</code>	Any whitespace

Example — extracting a phone number

```
\d{3}-\d{3}-\d{4}
```

Input text

“Call me at **415-555-2671** tomorrow.”

Match found

415-555-2671

✓ matches pattern

Quantifiers — How Many Times?

Quantifier	Meaning	Example
*	0 or more	ab* → a, ab, abbb
+	1 or more	ab+ → ab, abbb
?	0 or 1 (optional)	colou?r → color, colour
{n}	Exactly n	\d{4} → 2026
{n,m}	Between n and m	\d{2,4} → 12, 123, 1234

The “?” makes the “u” optional

```
colou?r
```

color ✓

colour ✓

Example — {n,m} range

```
\d{2,4}
```

12 ✓

123 ✓

1234 ✓

Anchors & Word Boundaries

Anchor	Meaning
<code>^</code>	Start of the string
<code>\$</code>	End of the string
<code>\b</code>	A word boundary (edge of a word)

Example — `^Hello` (start of string)

```
^Hello
```

`"Hello world"` ✓

`"Say Hello"` ✗

Example — `\bcat\b` (whole word only)

```
\bcat\b
```

`"The cat sat"` ✓

`"category"` ✗

`\b` stops “cat” from matching inside a longer word like “category.”

Example: Extracting Email Addresses

Pattern

```
[\\w.-]+@[\\w.-]+\\.\\w+
```

Input text

“Contact us at **support@nlp-tools.com** or **sales@example.org** for help.”

```
re.findall(pattern, text)
```

Result:

```
support@nlp-tools.com
```

```
sales@example.org
```

The pattern grabs everything shaped like username@domain.suffix — no dictionary of addresses needed.

WORKED EXAMPLE

Example: Tokenizing a Sentence

Pattern

```
\w+|[\^\w\s]
```

Matches a run of word characters, OR a single non-word / non-space character (punctuation).

Input sentence

```
"NLP is amazing, isn't it?"
```

Output tokens

NLP

is

amazing

,

isn

'

t

it

?

9 tokens: words and punctuation are split apart, and the contraction "isn't" breaks at the apostrophe.

Example: Cleaning Noisy Text

Pattern (used with re.sub to remove tags)

```
re.sub(r"<.*?>", "", text)
```

BEFORE

<p>Hello World!</p>



AFTER

Hello World!

"<.?>" matches anything between < and >. The "?" keeps it non-greedy, so each tag is matched separately instead of swallowing the whole line.*

Groups & Capturing

Parentheses () group part of a pattern and capture the text they match, so you can pull out just that piece.

Pattern

```
(\d{3}) - (\d{3} - \d{4})
```

Input: "415-555-2671"

Group 1 (area code)

415

Group 2 (local number)

555-2671

```
m = re.match(pattern, text)
m.group(1)    # "415"
m.group(2)    # "555-2671"
```

Common NLP Regex Patterns

Task	Pattern	Example match
Email address	<code>[\w.-]+@[\w.-]+\.\w+</code>	user@site.com
URL	<code>https?:\/\/\S+</code>	https://nlp.dev
Hashtag	<code>#\w+</code>	#NLProc
Date (MM/DD/YYYY)	<code>\d{1,2}\/\d{1,2}\/\d{4}</code>	09/07/2026
Number (incl. decimals)	<code>-?\d+\.\?\d*</code>	-3.14
Collapse whitespace	<code>\s+</code>	many spaces → one

Python's re Module — Quick Reference

```
re.match(pattern, text)
```

Checks for a match only at the start of the string

```
re.search(pattern, text)
```

Finds the first match anywhere in the string

```
re.findall(pattern, text)
```

Returns a list of all matches

```
re.sub(pattern, repl, text)
```

Replaces matches with new text

```
re.split(pattern, text)
```

Splits the string wherever the pattern matches

```
>>> re.findall(r'\d+', "I have 2 cats and 5 dogs")  
['2', '5']
```

Key Takeaways

1

A regex is a compact language for describing the shape of text — not exact words.

2

In NLP it powers preprocessing: tokenizing, cleaning noise, and extracting entities like emails, dates, and hashtags.

3

A small toolkit covers most needs: `.` `*` `+` `?` `[ ]` `\d` `\w` `\s` `^` `$` `\b` and `( )` for grouping.

4

Always test patterns on real, messy examples — edge cases (like "isn't" or nested tags) reveal gaps fast.

Tip: try patterns interactively at regex101.com before dropping them into a pipeline.



Thank You

Questions about regular expressions in NLP?